

Computación Paralela

Big Data Aplicado

Francisco Pascual Romero
FranciscoP.Romero@uclm.es



Big_Data
Aplicado



Curso Especialización
Inteligencia_Artificial y
Big_Data

Computación Paralela

Necesidad

- La **ley de Moore** expresa que aproximadamente cada 2 años se duplica el número de transistores en un microprocesador.
- Cada vez es más difícil cumplirla, y las empresas se acercan a los límites del silicio.



Computación paralela

¿Qué es?

- Permite el desarrollo de aplicaciones que aprovechen la utilización de:
 - múltiples procesadores
 - de forma colaborativa
 - para resolver un problema.
- Reducir el **tiempo** de ejecución de una aplicación mediante el uso de múltiples procesadores.
- Resolver problemas de mayor **tamaño**.



Computación Paralela

Particionamiento de Tareas

- Dividir una tarea grande en un número de diferentes subtareas que puedan ser complementadas por diferentes unidades de proceso.
- A pesar de que cada proceso realice una subtaska será necesario que estos se comuniquen para poder cooperar en la obtención de la solución global del problema.



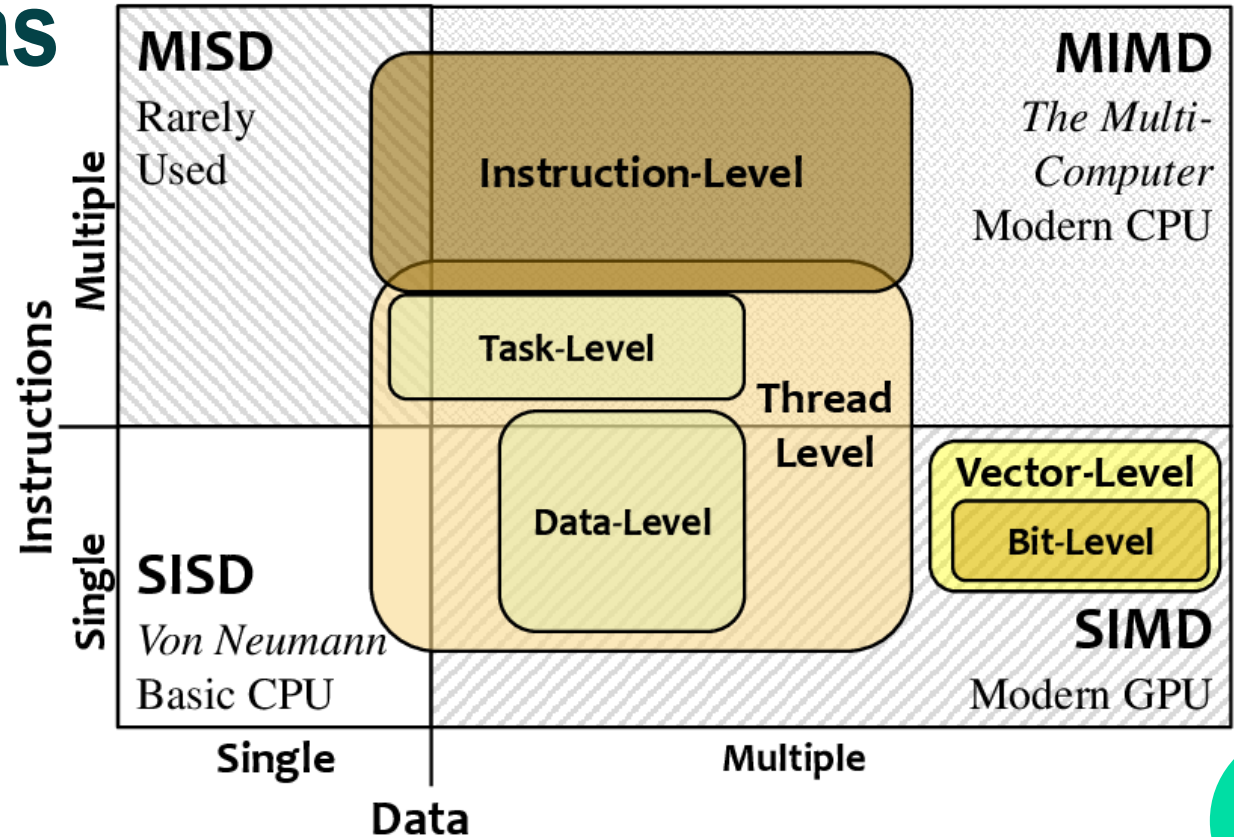
Paralelismo

- A nivel de **tareas** el trabajo se reparte en varias tareas que se distribuyen por los diferentes núcleos o procesadores.
- A nivel de **datos**: los datos se dividen para resolver el problema entre los diferentes núcleos que realizan operaciones similares sobre los datos.



Arquitecturas Paralelas

Taxonomía de Flynn



Paralelismo

- A nivel de **instrucción** (ILP) ejecuta las instrucciones en varios núcleos de procesamiento para aumentar el rendimiento.
- A nivel de **hilo (multi-hilo)**: asigna múltiples flujos de ejecución en varios núcleo.
 - **SIMD**: lotes de hilos (warps) se asignan a lotes de núcleos de procesamiento.
- Otros: a nivel de bit, etc.



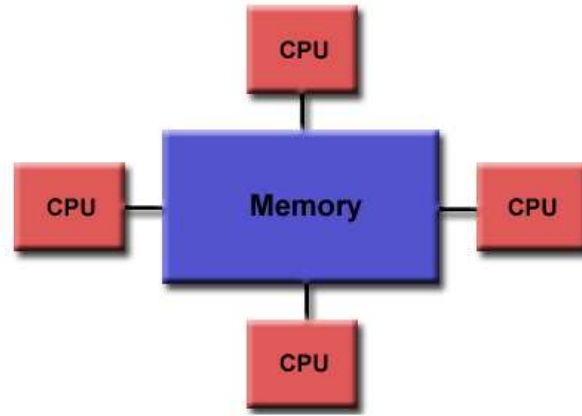
Programación Paralela

- La arquitectura subyacente tiene una gran importancia para maximizar el rendimiento
- Uso de herramientas de análisis (profilers) para saber dónde optimizar y paralelizar.
- Memoria distribuida: MPI
- Memoria compartida: OpenMP



Programación Paralela

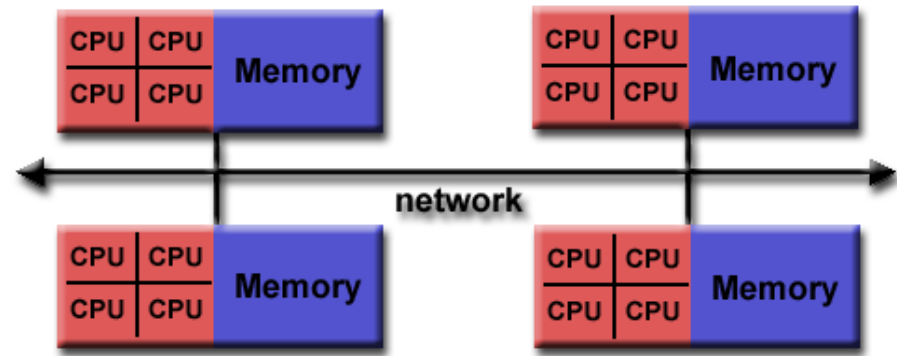
Memoria Compartida



- Los procesos (multihilo) acceden directamente a toda la memoria a pesar de que este en diferentes sitios físicos
- Acceso uniforme y No uniforme en tiempo.

Programación Paralela

Memoria Distribuida

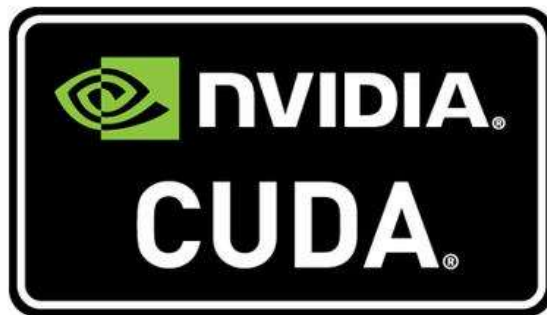


- La memoria ya que no es una imagen única sino que es de cada procesador.
- Procesadores deslocalizados no hay acceso uniforme a memoria.
- La comunicación es un factor crítico para la eficiencia

Programación Paralela

Otros Modelos

- **Mixtas:** MPI + OpenMP
- **Cuda** para GPUs Nvidia (SIMD)
- **OpenCL:** Paralelismo de datos GPU + CPU
- **OpenACC:** Simplificar la programación paralela de sistemas heterogéneos de CPU/GPU.1



Escalabilidad

- Una tecnología es escalable si es capaz de manejar un problema de tamaño creciente.
 - Un programa es escalable si mantiene la eficiencia independiente del tamaño del problema.
 - La capacidad de que alguna aplicación aumente su velocidad cuando se incrementan recursos.
 - La capacidad para que alguna aplicación resuelva problemas más grandes cuando los recursos aumentan.
- 

Escalabilidad

- ¿Cuánto más rápido puede resolverse un determinado problema con N procesadores en lugar de uno?
 - ¿Cuánto más trabajo se puede hacer con N procesadores en lugar de uno?
 - ¿Qué impacto tienen los requisitos de comunicación de la aplicación paralela en el rendimiento?
 - ¿Qué fracción de los recursos se utiliza realmente de forma productiva para resolver el problema?
- 

Escalabilidad

Tipos

- **Escalado Fuerte:**

- Tamaño de problema fijo
- Ponemos más capacidad, repartimos el mismo trabajo entre más.
- Minimizamos el tiempo en resolver un problema dado.

- **Escalado Débil:**

- Como hay más procesadores se pueden procesar más datos.
 - Intentar resolver problemas más grandes.
- 

Ley de Amdahl

- Teóricamente si uno dobla el número de procesadores, el tiempo de ejecución debería reducirse a la mitad.
- Todo programa consta de partes paralelizables y partes no paralelizables.
- La ley de Amdahl describe la relación entre la aceleración esperada de la implementación paralela de un algoritmo y su implementación secuencial.

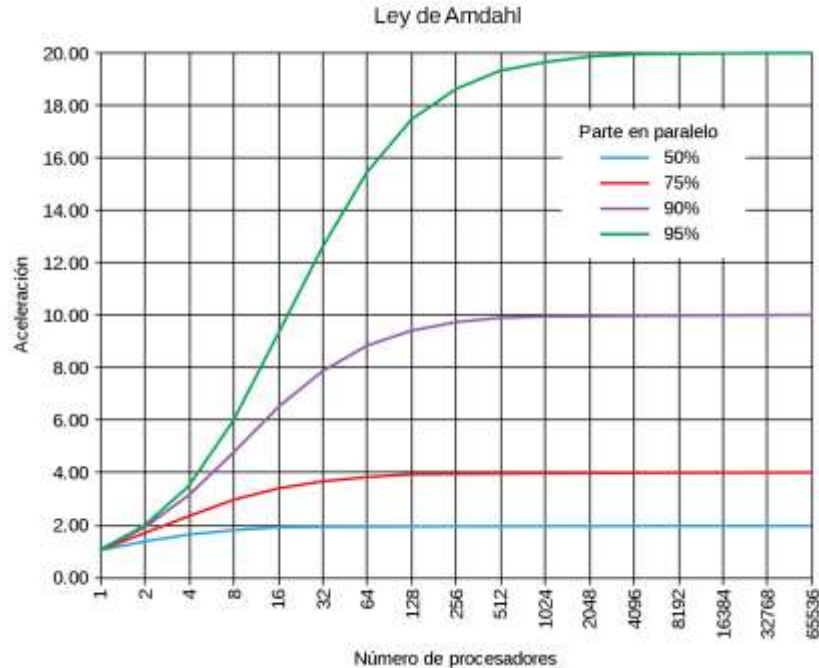
Ley de Amdahl

- la aceleración **S** que se puede alcanzar a partir de las modificaciones (mejoras) de una porción **P** de un cálculo considerando el número de procesadores **N**.

$$S = \frac{1}{(1 - P) + \frac{P}{N}}$$

Ley de Amdahl

- La aceleración de un programa paralelo está limitada por la porción secuencial del mismo.



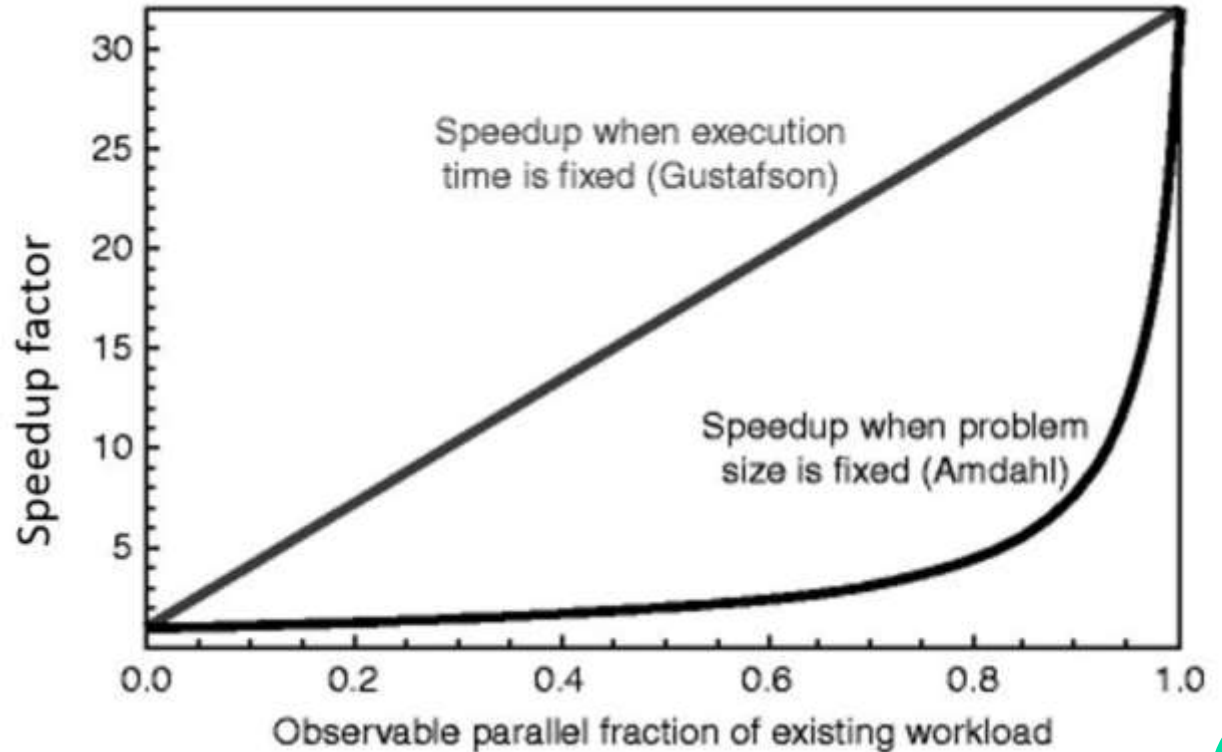
Ley de Gustaffson

- La parte paralela de un programa decrece cuando el tamaño del problema aumentan.
- Cualquier problema suficientemente grande puede ser eficientemente paralelizado.

$$S(P) = P - \alpha \cdot (P - 1)$$

- **P** es el número de procesadores, **S** es el speedup, y α la parte no paralelizable del proceso.

Ley de Gustaffson



Computación Paralela

Big Data Aplicado

Francisco Pascual Romero
FranciscoP.Romero@uclm.es



Big_Data
Aplicado



Curso Especialización
Inteligencia_Artificial y
Big_Data